

d)

1 - Luego de realizado el ejercicio agregar al código para la posibilidad de cargar 50000 clientes y 50000 artículos. ¿Qué campos considera el Equipo que pueden considerarse índices y dar un ejemplo de consulta en JPAQL utilizando esos campos índices y consulta sin utilizar índice? Analizar el tiempo de respuesta y describir las conclusiones que obtuvo el equipo.

Los campos que se podrían considerar índices podrían ser:

- Para la clase Cliente el dni por ejemplo ya que es un identificador único también y cada persona conoce el suyo, entonces para consultas y búsquedas sería una manera rápida aparte del identificador propio que asigna la base de datos de manera automática.
- Para la clase Artículo podría ser la denominación por la misma razón mencionada anteriormente, es un identificador único (si solamente le damos opciones a los usuarios y no permitimos que cada uno agregue el suyo propio para evitar confusiones)

@Entity

@Table(name = "cliente", indexes = { @Index(name = "idx_cliente_dni", columnList = "dni", unique = true) })

public class Cliente { ... }

@Entity

@Table(name = "articulo", indexes = { @Index(name = "idx_articulo_denominacion", columnList = "denominacion") })

public class Artículo { ... }

EJEMPLO CONSULTA JPA SIN INDICES SOBRE CLIENTE

```
int dniABuscar = 20048500;
long inicioSinIndice = System.nanoTime();

Cliente clienteSinIndice = em.createQuery(
    "SELECT c FROM Cliente c WHERE c.dni = :dni", Cliente.class)
    .setParameter("dni", dniABuscar)
    .getSingleResult();

long finSinIndice = System.nanoTime();
double tiempoSinIndiceMs = (finSinIndice - inicioSinIndice) / 1_000_000.0;

System.out.println("=====");
System.out.println("RESULTADOS DE RENDIMIENTO (50.000 REGISTROS)");
System.out.println("=====");
System.out.println("Tiempo sin índice:      " + tiempoSinIndiceMs + " ms");
```

```
=====
RESULTADOS DE RENDIMIENTO (50.000 REGISTROS)
=====
Tiempo sin índice:      242.532947 ms
ago 16, 2026 3:43:09 P. M. org.hibernate.engine.jd
```

EJEMPLO CONSULTA JPA CON INDICES SOBRE CLIENTE

```
int dniABuscar = 20048500;
long inicioConIndice = System.nanoTime();

Cliente clienteConIndice = em.createQuery(
    "SELECT c FROM Cliente c WHERE c.dni = :dni", Cliente.class)
    .setParameter("dni", dniABuscar)
    .getSingleResult();

long finConIndice = System.nanoTime();
double tiempoConIndiceMs = (finConIndice - inicioConIndice) / 1_000_000.0;

System.out.println("=====");
System.out.println("RESULTADOS DE RENDIMIENTO (50.000 REGISTROS)");
System.out.println("=====");
System.out.println("Tiempo con índice:      " + tiempoConIndiceMs + " ms");
```

```
=====
RESULTADOS DE RENDIMIENTO (50.000 REGISTROS)
=====
Tiempo con índice:      190.914184 ms
ago 16, 2026 4:01:09 P. M. org.hibernate.engine.jd
```

Podemos observar la reducción drástica de tiempo de respuesta ya que, buscando el mismo cliente sin índice en el DNI necesitamos de 242.53ms mientras que con el índice fue de 186.87 ms dándonos así, una diferencia de 51.62 ms menos por consulta (un 21.28% de mejora en el tiempo de respuesta). Si lo pasamos a factor velocidad (Speedup), el tiempo de mejora es de aproximadamente 1.27x a favor de la consulta indexada.

2 - ¿Cuál sentencia de SQL, teniendo en cuenta la base de datos utilizada, el Equipo puede tener en cuenta para conocer el costo (tiempo de respuesta) que tiene una consulta a la base de datos?

La base de datos utilizada fue MySQL y para ella la sentencia clave es EXPLAIN/EXPLAIN ANALYZE:

```
EXPLAIN ANALYZE SELECT * FROM cliente WHERE dni = 20045000;
```

EXPLAIN nos devuelve el tiempo de ejecución estático (si usó un índice type: ref/const o si hizo un escaneo completo type: ALL y la cantidad de filas leídas).

EXPLAIN ANALYZE: ejecuta la consulta real y muestra el costo estimado del optimizador, el tiempo real en milisegundos de cada etapa y el número exacto de filas procesadas.

3 - ¿Cuál es la configuración en JPA que se puede utilizar para conocer cómo el motor de persistencia transforma los JPQL en sentencia SQL? Dar un ejemplo del JPAQL para la obtención de Factura y como queda esa sentencia expresada en SQL.

La configuración de JPA para conocer cómo se transforman las consultas JPQL en sentencias de SQL que hace el ORM (en nuestro caso hibernate) son:

```
<!-- Muestra el SQL en la consola -->
<property name="hibernate.show_sql" value="true" />
<!-- Formatea el SQL para que sea legible -->
<property name="hibernate.format_sql" value="true" />
<!-- Resalta las palabras clave SQL con colores -->
<property name="hibernate.highlight_sql" value="true" />
```

Estas tres configuraciones se colocan en un archivo llamado persistence.xml.

Ejemplo:

```
Factura factura = em.createQuery(
    "SELECT f FROM Factura f WHERE f.numero = :numero", Factura.class)
    .setParameter("numero", 10)
    .getSingleResult();
```

Sentencia SQL:

```
SELECT f1_0.id,
f1_0.fecha,
f1_0.id_cliente,
f1_0.numero,
f1_0.total
FROM factura f1_0 WHERE f1_0.numero = ?;
```